In [1]:
```
#importing libraries
```

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [3]:
```
#loading dataset
```

In [4]:
```
df = pd.read_csv("Sample - Superstore.csv")
```

In [5]:
```
#understanding dataset
```

In [6]:
```
df.head()
```

Out[6]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Countr
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	Unite Stat
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	Unite Stat
2	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	Unite Stat
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	Unite Stat
4	5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	Unite Stat

5 rows × 21 columns

In [7]:
```
df.shape
```

Out[7]:
```
(9994, 21)
```

In [8]:
```
df.info
```

Out[8]:
```
<bound method DataFrame.info of       Row ID        Order ID  Order Date   Ship Da
te       Ship Mode  \
0          1  CA-2016-152156   11/8/2016  11/11/2016    Second Class
1          2  CA-2016-152156   11/8/2016  11/11/2016    Second Class
2          3  CA-2016-138688   6/12/2016   6/16/2016    Second Class
3          4  US-2015-108966  10/11/2015  10/18/2015  Standard Class
4          5  US-2015-108966  10/11/2015  10/18/2015  Standard Class
...      ...             ...         ...         ...             ...
9989    9990  CA-2014-110422   1/21/2014   1/23/2014    Second Class
9990    9991  CA-2017-121258   2/26/2017    3/3/2017  Standard Class
9991    9992  CA-2017-121258   2/26/2017    3/3/2017  Standard Class
9992    9993  CA-2017-121258   2/26/2017    3/3/2017  Standard Class
9993    9994  CA-2017-119914    5/4/2017    5/9/2017    Second Class

     Customer ID     Customer Name    Segment        Country             City  \
0       CG-12520       Claire Gute   Consumer  United States        Henderson
1       CG-12520       Claire Gute   Consumer  United States        Henderson
2       DV-13045   Darrin Van Huff  Corporate  United States      Los Angeles
3       SO-20335    Sean O'Donnell   Consumer  United States  Fort Lauderdale
4       SO-20335    Sean O'Donnell   Consumer  United States  Fort Lauderdale
...          ...               ...        ...            ...              ...
9989    TB-21400  Tom Boeckenhauer   Consumer  United States            Miami
9990    DB-13060       Dave Brooks   Consumer  United States       Costa Mesa
9991    DB-13060       Dave Brooks   Consumer  United States       Costa Mesa
9992    DB-13060       Dave Brooks   Consumer  United States       Costa Mesa
9993    CC-12220      Chris Cortes   Consumer  United States      Westminster

      ... Postal Code  Region       Product ID         Category Sub-Category  \
0     ...       42420   South  FUR-BO-10001798        Furniture    Bookcases
1     ...       42420   South  FUR-CH-10000454        Furniture       Chairs
2     ...       90036    West  OFF-LA-10000240  Office Supplies       Labels
3     ...       33311   South  FUR-TA-10000577        Furniture       Tables
4     ...       33311   South  OFF-ST-10000760  Office Supplies      Storage
...   ...         ...     ...              ...              ...          ...
9989  ...       33180   South  FUR-FU-10001889        Furniture  Furnishings
9990  ...       92627    West  FUR-FU-10000747        Furniture  Furnishings
9991  ...       92627    West  TEC-PH-10003645       Technology       Phones
9992  ...       92627    West  OFF-PA-10004041  Office Supplies        Paper
9993  ...       92683    West  OFF-AP-10002684  Office Supplies   Appliances

                                          Product Name     Sales  Quantity  \
0                    Bush Somerset Collection Bookcase  261.9600         2
1     Hon Deluxe Fabric Upholstered Stacking Chairs,...  731.9400         3
2     Self-Adhesive Address Labels for Typewriters b...   14.6200         2
3         Bretford CR4500 Series Slim Rectangular Table  957.5775         5
4                       Eldon Fold 'N Roll Cart System   22.3680         2
...                                                ...       ...       ...
9989                            Ultra Door Pull Handle   25.2480         3
9990  Tenex B1-RE Series Chair Mats for Low Pile Car...   91.9600         2
9991                             Aastra 57i VoIP phone  258.5760         2
9992  It's Hot Message Books with Stickers, 2 3/4" x 5"   29.6000         4
9993  Acco 7-Outlet Masterpiece Power Center, Wihtou...  243.1600         2

      Discount    Profit
0         0.00   41.9136
1         0.00  219.5820
```

```
2         0.00    6.8714
3         0.45 -383.0310
4         0.20    2.5164
...        ...       ...
9989      0.20    4.1028
9990      0.00   15.6332
9991      0.20   19.3932
9992      0.00   13.3200
9993      0.00   72.9480

[9994 rows x 21 columns]>
```

In [9]:
```
df.describe()
```

Out[9]:

	Row ID	Postal Code	Sales	Quantity	Discount	Profit
count	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000
mean	4997.500000	55190.379428	229.858001	3.789574	0.156203	28.656896
std	2885.163629	32063.693350	623.245101	2.225110	0.206452	234.260108
min	1.000000	1040.000000	0.444000	1.000000	0.000000	-6599.978000
25%	2499.250000	23223.000000	17.280000	2.000000	0.000000	1.728750
50%	4997.500000	56430.500000	54.490000	3.000000	0.200000	8.666500
75%	7495.750000	90008.000000	209.940000	5.000000	0.200000	29.364000
max	9994.000000	99301.000000	22638.480000	14.000000	0.800000	8399.976000

In [10]:
```
df.columns
```

Out[10]:
```
Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode',
       'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State',
       'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category',
       'Product Name', 'Sales', 'Quantity', 'Discount', 'Profit'],
      dtype='object')
```

In [11]:
```
#renaming some columns
```

In [12]:
```
df.columns=df.columns.str.replace(' ', '_')
```

In [13]:
```
df.columns
```

Out[13]:
```
Index(['Row_ID', 'Order_ID', 'Order_Date', 'Ship_Date', 'Ship_Mode',
       'Customer_ID', 'Customer_Name', 'Segment', 'Country', 'City', 'State',
       'Postal_Code', 'Region', 'Product_ID', 'Category', 'Sub-Category',
       'Product_Name', 'Sales', 'Quantity', 'Discount', 'Profit'],
      dtype='object')
```

In [14]:
```
df.columns=df.columns.str.replace('ID', 'Id')
```

In [15]:
```
df.columns
```

Out[15]:
```
Index(['Row_Id', 'Order_Id', 'Order_Date', 'Ship_Date', 'Ship_Mode',
       'Customer_Id', 'Customer_Name', 'Segment', 'Country', 'City', 'State',
       'Postal_Code', 'Region', 'Product_Id', 'Category', 'Sub-Category',
       'Product_Name', 'Sales', 'Quantity', 'Discount', 'Profit'],
      dtype='object')
```

In [16]:
```
#checking missing values
```

In [17]:
```
df.isnull().sum()
```

Out[17]:
```
Row_Id           0
Order_Id         0
Order_Date       0
Ship_Date        0
Ship_Mode        0
Customer_Id      0
Customer_Name    0
Segment          0
Country          0
City             0
State            0
Postal_Code      0
Region           0
Product_Id       0
Category         0
Sub-Category     0
Product_Name     0
Sales            0
Quantity         0
Discount         0
Profit           0
dtype: int64
```

In [18]:
```
#checking duplicates
```

In [19]:
```
df.duplicated()
```

Out[19]:
```
0       False
1       False
2       False
3       False
4       False
        ...
9989    False
9990    False
9991    False
9992    False
9993    False
Length: 9994, dtype: bool
```

In [20]:
```
#checking data types
```

In [21]:
```
df.dtypes
```

Out[21]:
```
Row_Id             int64
Order_Id          object
Order_Date        object
Ship_Date         object
Ship_Mode         object
Customer_Id       object
Customer_Name     object
Segment           object
Country           object
City              object
State             object
Postal_Code        int64
Region            object
Product_Id        object
Category          object
Sub-Category      object
Product_Name      object
Sales            float64
Quantity           int64
Discount         float64
Profit           float64
dtype: object
```

In [22]:
```
#changing some datatypes
```

In [23]:
```
df['Order_Date'] = pd.to_datetime(df['Order_Date'])
df['Ship_Date'] = pd.to_datetime(df['Ship_Date'])
```

In [24]:
```
#analyzing some columns
```

In [25]:
```
df['Ship_Mode'].value_counts()
```

Out[25]:
```
Ship_Mode
Standard Class    5968
Second Class      1945
First Class       1538
Same Day           543
Name: count, dtype: int64
```

In [26]:
```
df['Segment'].value_counts()
```

Out[26]:
```
Segment
Consumer       5191
Corporate      3020
Home Office    1783
Name: count, dtype: int64
```

In [27]:
```
df['Country'].value_counts()
```

Out[27]:
```
Country
United States    9994
Name: count, dtype: int64
```

In [28]:
```
df['City'].value_counts()
```

Out[28]:
```
City
New York City      915
Los Angeles        747
Philadelphia       537
San Francisco      510
Seattle            428
                  ...
Glenview             1
Missouri City        1
Rochester Hills      1
Palatine             1
Manhattan            1
Name: count, Length: 531, dtype: int64
```

In [29]:
```
df['State'].value_counts()
```

Out[29]:
```
State
California              2001
New York                1128
Texas                    985
Pennsylvania             587
Washington               506
Illinois                 492
Ohio                     469
Florida                  383
Michigan                 255
North Carolina           249
Arizona                  224
Virginia                 224
Georgia                  184
Tennessee                183
Colorado                 182
Indiana                  149
Kentucky                 139
Massachusetts            135
New Jersey               130
Oregon                   124
Wisconsin                110
Maryland                 105
Delaware                  96
Minnesota                 89
Connecticut               82
Oklahoma                  66
Missouri                  66
Alabama                   61
Arkansas                  60
Rhode Island              56
Utah                      53
Mississippi               53
Louisiana                 42
South Carolina            42
Nevada                    39
Nebraska                  38
New Mexico                37
Iowa                      30
New Hampshire             27
Kansas                    24
Idaho                     21
Montana                   15
South Dakota              12
Vermont                   11
District of Columbia      10
Maine                      8
North Dakota               7
West Virginia              4
Wyoming                    1
Name: count, dtype: int64
```

In [30]:
```
df['Region'].value_counts()
```

Out[30]:
```
Region
West       3203
East       2848
Central    2323
South      1620
Name: count, dtype: int64
```

In [31]:
```
df.dtypes
```

Out[31]:
```
Row_Id                    int64
Order_Id                 object
Order_Date       datetime64[ns]
Ship_Date        datetime64[ns]
Ship_Mode                object
Customer_Id              object
Customer_Name            object
Segment                  object
Country                  object
City                     object
State                    object
Postal_Code               int64
Region                   object
Product_Id               object
Category                 object
Sub-Category             object
Product_Name             object
Sales                   float64
Quantity                  int64
Discount                float64
Profit                  float64
dtype: object
```

In [32]:
```
#changing id columns to category
```

In [33]:
```
df['Order_Id'] = df['Order_Id'].astype('category')
df['Customer_Id'] = df['Customer_Id'].astype('category')
df['Product_Id'] = df['Product_Id'].astype('category')
```

In [34]:
```
#changing columns to categoryy together
```

In [35]:
```
cat_cols = ['Ship_Mode', 'Segment', 'Country', 'City',
            'State', 'Region', 'Category', 'Sub-Category']

for col in cat_cols:
    df[col] = df[col].astype('category')
```

In [36]:
```
#changing postal code to object
```

In [37]:
```
df['Postal_Code'] = df['Postal_Code'].astype('str')
```

In [38]:
```
df.dtypes
```

Out[38]:
```
Row_Id                    int64
Order_Id               category
Order_Date       datetime64[ns]
Ship_Date        datetime64[ns]
Ship_Mode              category
Customer_Id            category
Customer_Name            object
Segment                category
Country                category
City                   category
State                  category
Postal_Code              object
Region                 category
Product_Id             category
Category               category
Sub-Category           category
Product_Name             object
Sales                   float64
Quantity                  int64
Discount                float64
Profit                  float64
dtype: object
```

In [39]:
```
#making new columns for analysis
```

In [40]:
```
df['Year'] = df['Order_Date'].dt.year
df['Month'] = df['Order_Date'].dt.month
df['Month_Name'] = df['Order_Date'].dt.month_name()
```

In [41]:
```
#checking some values
```

In [42]:
```
total_sales = df['Sales'].sum()
total_profit = df['Profit'].sum()
avg_order_value = df['Sales'].mean()
total_orders = df['Order_Id'].nunique()
```

In [43]:
```
total_sales, total_profit, avg_order_value, total_orders
```

Out[43]:
```
(np.float64(2297200.8603),
 np.float64(286397.0217),
 np.float64(229.85800083049827),
 5009)
```

In [44]:
```
#grouping column region by sales
```

In [45]:
```
top_region = df.groupby('Region',observed='True')['Sales'].sum().sort_values(ascend
```

Data Analysis

In [46]:
```
#grouping sales by category
```

In [47]:
```
sales_category = df.groupby('Category',observed=True)['Sales'].sum().sort_values(as
```

In [48]:
```
#grouping sales by sub-category and knowing first 10 values
```

In [49]:
```
top_subcat = df.groupby('Sub-Category',observed=True)['Sales'].sum().sort_values(as
top_subcat.head(10)
```

Out[49]:
```
Sub-Category
Phones         330007.0540
Chairs         328449.1030
Storage        223843.6080
Tables         206965.5320
Binders        203412.7330
Machines       189238.6310
Accessories    167380.3180
Copiers        149528.0300
Bookcases      114879.9963
Appliances     107532.1610
Name: Sales, dtype: float64
```

In [50]:
```
#grouping sales by region
```

In [51]:
```
sales_region = df.groupby('Region',observed=True)['Sales'].sum()
sales_region
```

Out[51]:
```
Region
Central    501239.8908
East       678781.2400
South      391721.9050
West       725457.8245
Name: Sales, dtype: float64
```

In [52]:
```
#grouping profit by category
```

In [53]:
```
profit_category = df.groupby('Category',observed=True)['Profit'].sum()
profit_category
```

Out[53]:
```
Category
Furniture           18451.2728
Office Supplies    122490.8008
Technology         145454.9481
Name: Profit, dtype: float64
```

In [54]:
```
#checking loss making products and seeing first 5 products making loss
```

In [55]:
```
loss_products = df[df['Profit'] < 0]
loss_products.head()
```

Out[55]:

	Row_Id	Order_Id	Order_Date	Ship_Date	Ship_Mode	Customer_Id	Customer_Name
3	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell
14	15	US-2015-118983	2015-11-22	2015-11-26	Standard Class	HP-14815	Harold Pawlan
15	16	US-2015-118983	2015-11-22	2015-11-26	Standard Class	HP-14815	Harold Pawlan
23	24	US-2017-156909	2017-07-16	2017-07-18	Second Class	SF-20065	Sandra Flanagan
27	28	US-2015-150630	2015-09-17	2015-09-21	Standard Class	TB-21520	Tracy Blumstein

5 rows × 24 columns

In [56]:
```
#grouping sales by month
```

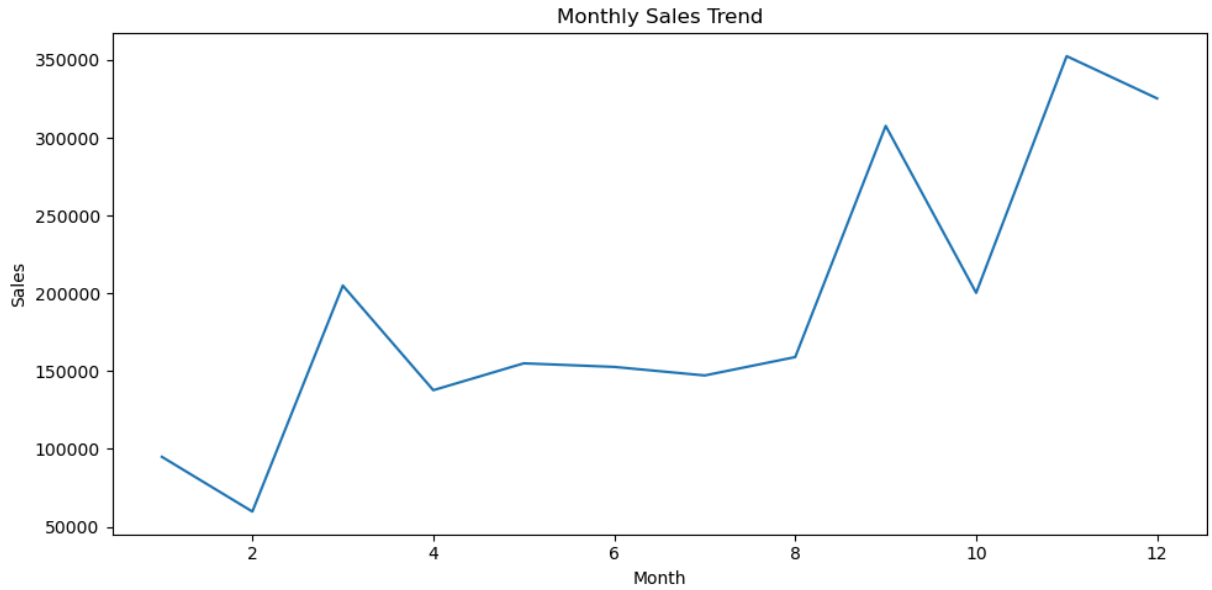
In [57]:
```
monthly_sales = df.groupby('Month')['Sales'].sum()
monthly_sales
```

Out[57]:
```
Month
1      94924.8356
2      59751.2514
3     205005.4888
4     137762.1286
5     155028.8117
6     152718.6793
7     147238.0970
8     159044.0630
9     307649.9457
10    200322.9847
11    352461.0710
12    325293.5035
Name: Sales, dtype: float64
```

Data Visualization

In [58]:
```
#plotting sales trend chart
```

In [59]:
```
plt.figure(figsize=(10,5))

plt.plot(monthly_sales.index, monthly_sales.values)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.tight_layout()
```

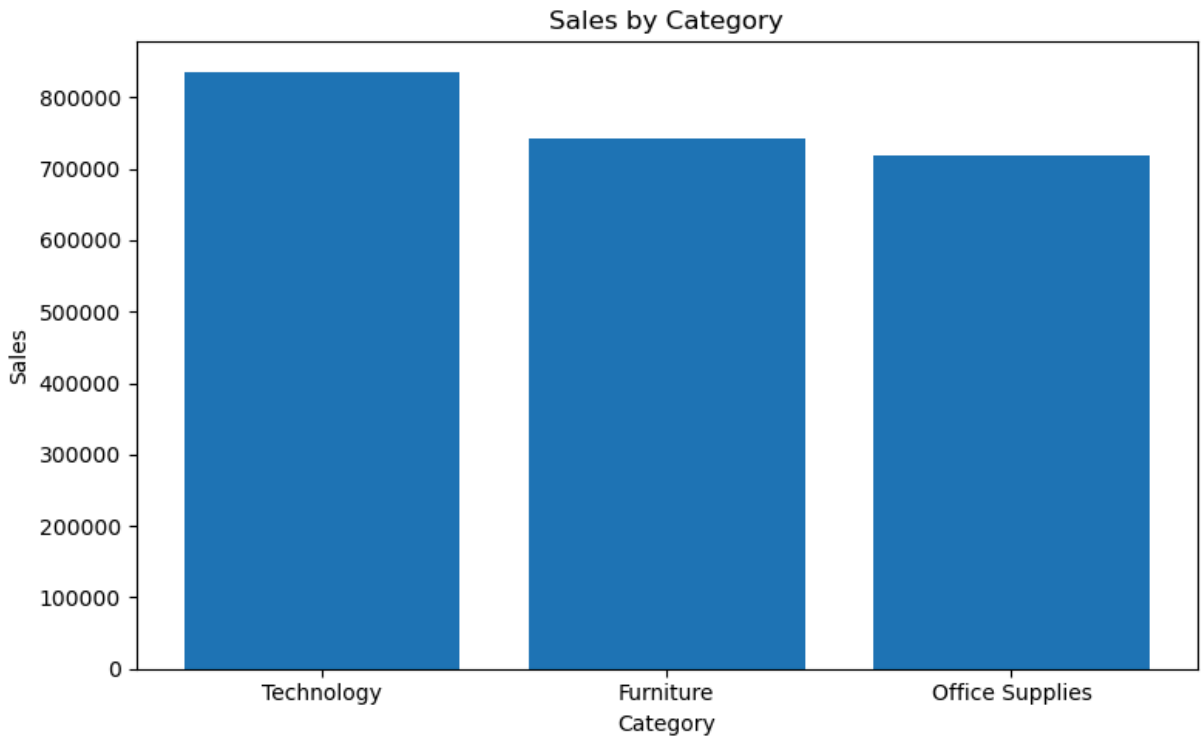


In [60]:
```
plt.savefig("sales_trend.png", bbox_inches='tight')
```

```
<Figure size 640x480 with 0 Axes>
```

In [61]:
```
#plotting sales by category
```

In [62]:
```
plt.figure(figsize=(8,5))

plt.bar(sales_category.index, sales_category.values)
plt.title('Sales by Category')
plt.xlabel("Category")
plt.ylabel("Sales")
plt.tight_layout()
plt.show()
```

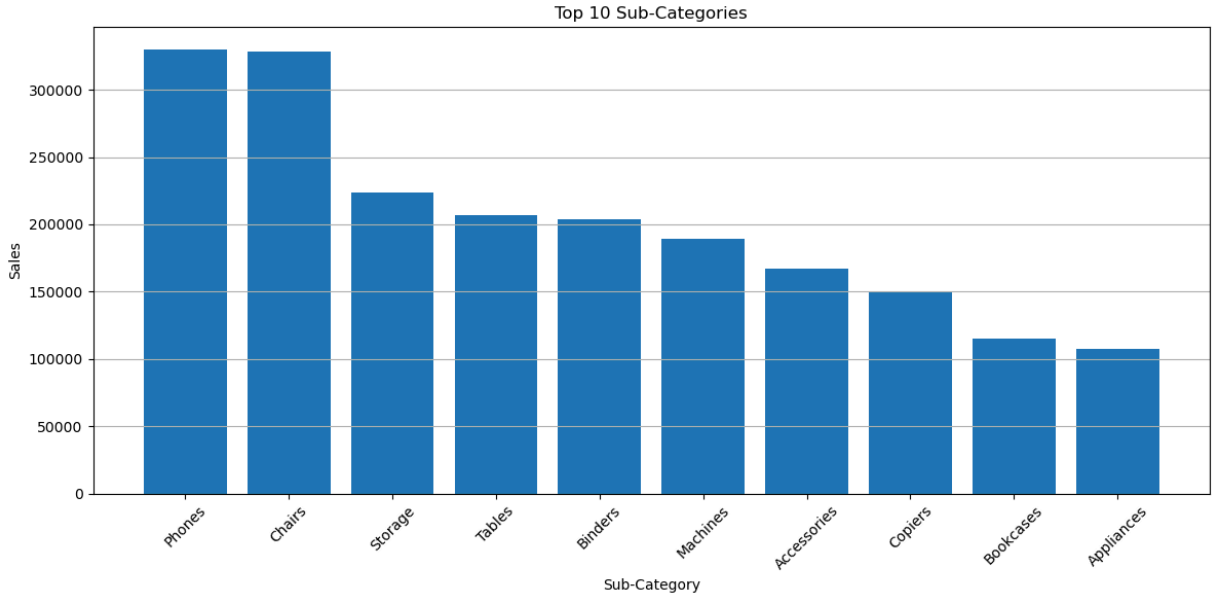


In [63]:
```
plt.savefig("sales_by_category.png", bbox_inches='tight')
```

```
<Figure size 640x480 with 0 Axes>
```

In [64]:
```
#plotting top 10 subcategory
```

In [65]:
```
plt.figure(figsize=(12,6))

top10 = top_subcat.head(10)
plt.bar(top10.index, top10.values)
plt.title("Top 10 Sub-Categories")
plt.xlabel("Sub-Category")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```

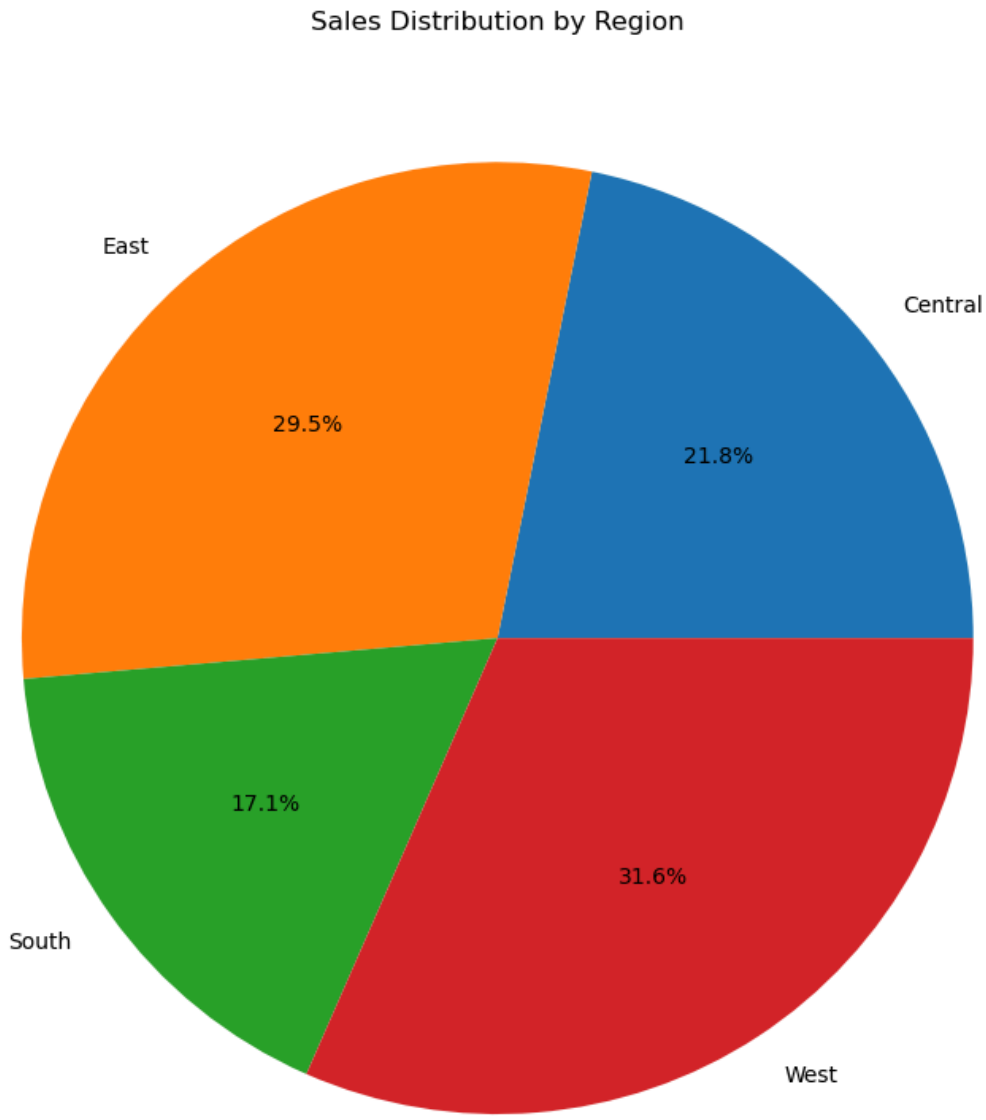


In [66]:
```
plt.savefig("top_10_subcategory.png", bbox_inches='tight')
```

```
<Figure size 640x480 with 0 Axes>
```

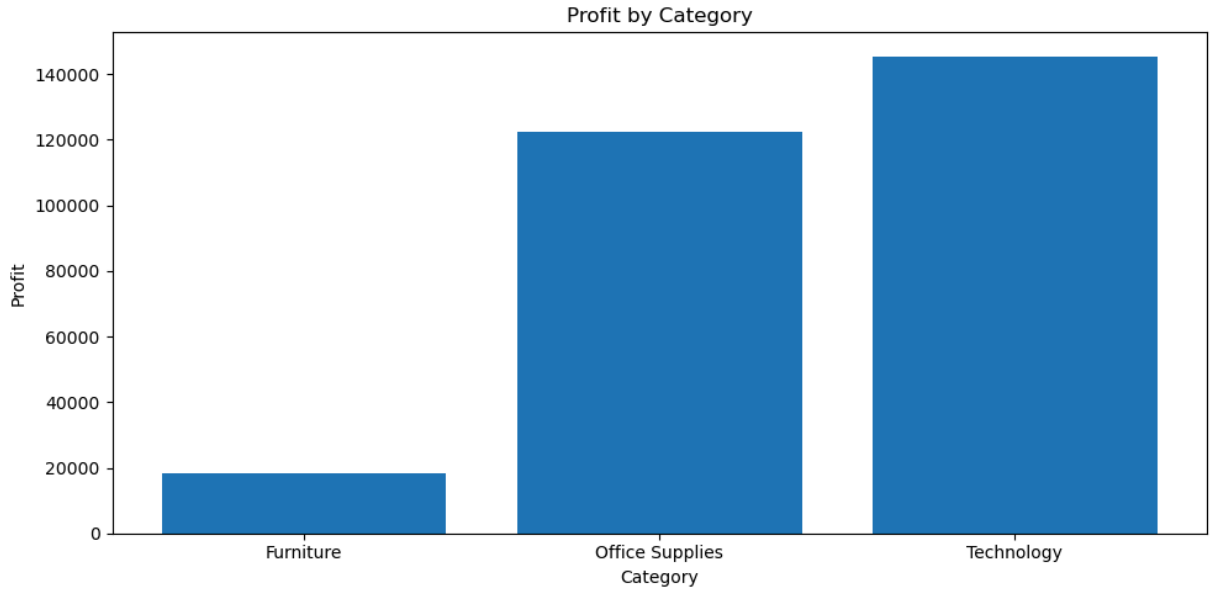
In [67]:
```
#plotting sales by region
```

In [68]:
```
plt.figure(figsize=(8,8))

plt.pie(
    sales_region.values,
    labels=sales_region.index,
    autopct='%1.1f%%'
)
plt.title("Sales Distribution by Region")
plt.tight_layout()
plt.savefig("sales_by_region.png", bbox_inches='tight')  # save BEFORE show
plt.show()
plt.close()
```



In [69]:
```
#plotting profit by category
```

In [70]:
```
plt.figure(figsize=(10,5))

plt.bar(profit_category.index, profit_category.values)
plt.title("Profit by Category")
plt.xlabel("Category")
plt.ylabel("Profit")
plt.tight_layout()
plt.savefig("profit_by_category.png", bbox_inches='tight')  # save BEFORE show
plt.show()
plt.close()
```



In [71]:
```
#plotting sales vs profit
```

In [72]:
```
sns.scatterplot(x='Sales', y='Profit', data=df)
plt.title("Sales vs Profit")
plt.tight_layout()
plt.savefig("sales_vs_profit.png", bbox_inches='tight')  # save BEFORE show
plt.show()
plt.close()
```

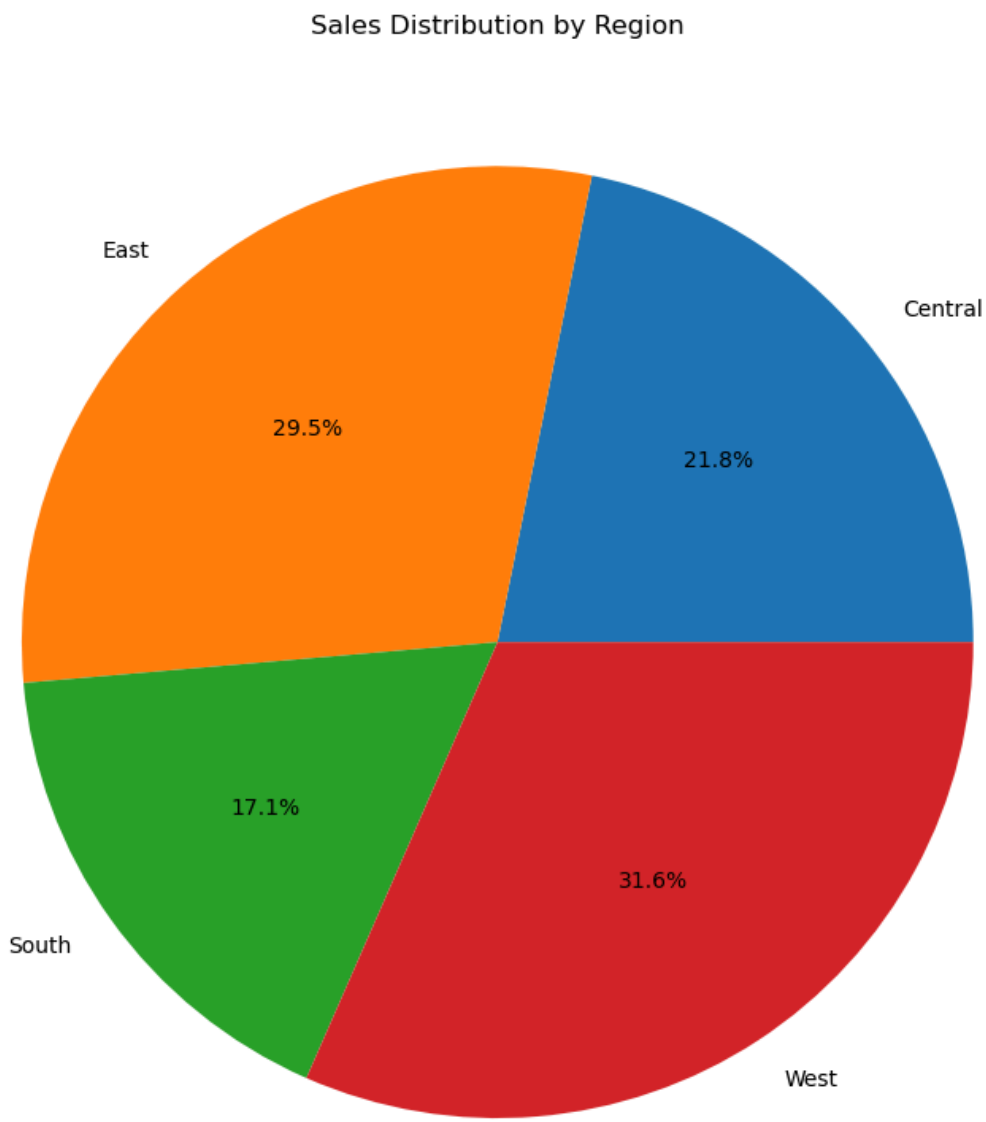
In [74]:
```
import os
print(os.getcwd())
```

```
C:\Users\HP
```

In [75]:
```
import os
print("sales_by_region.png" in os.listdir())
```

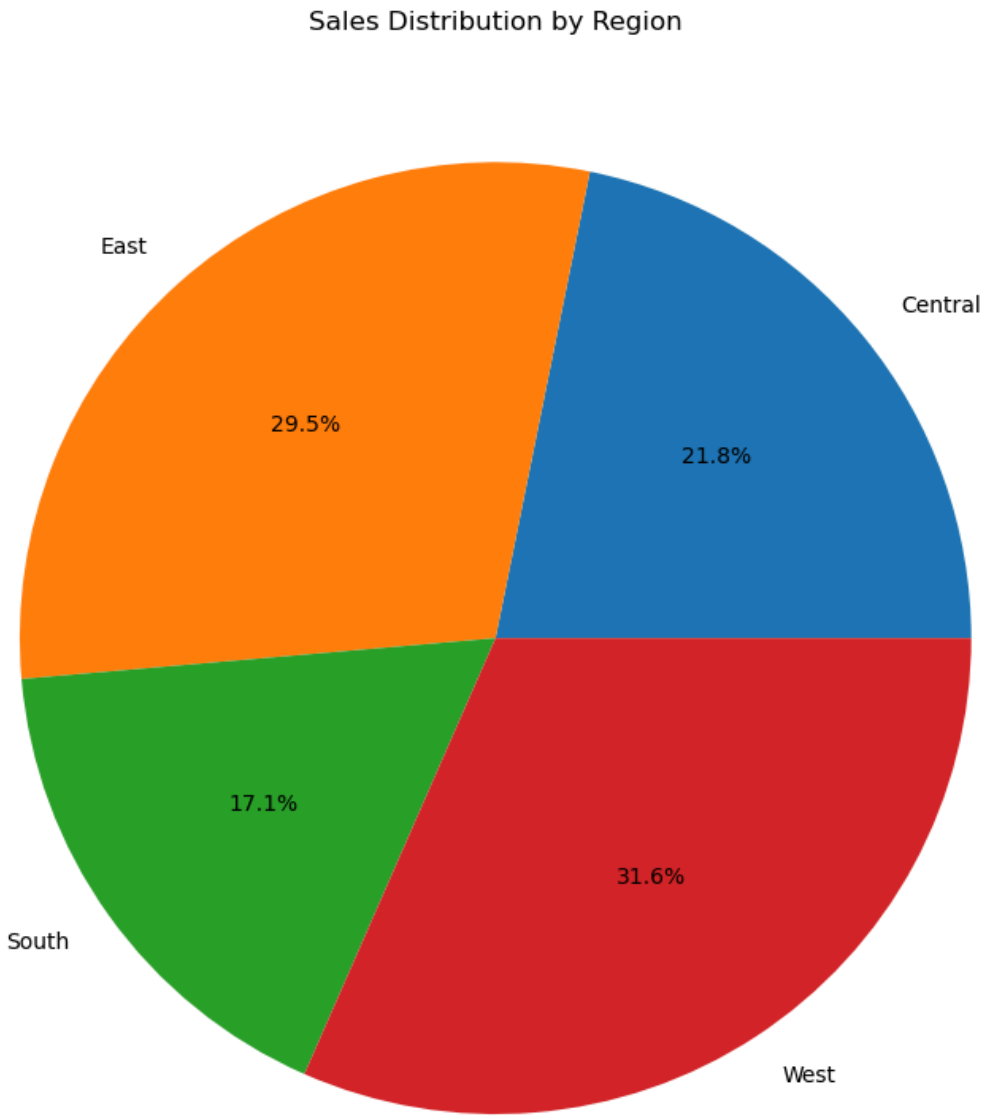
```
True
```

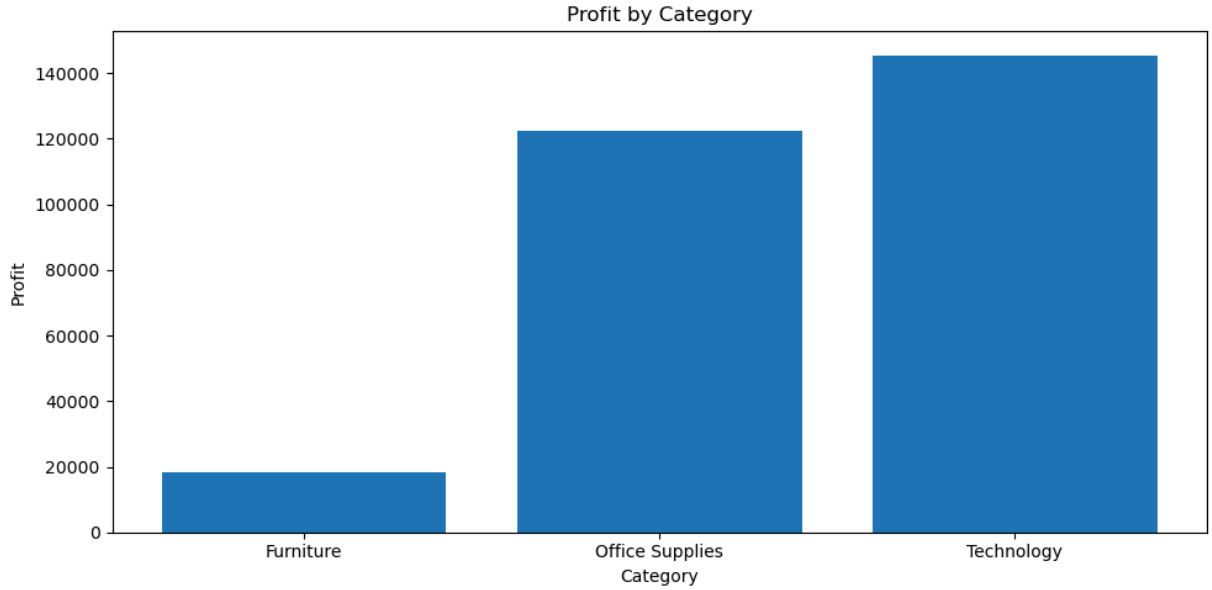
In [76]:
```
from IPython.display import Image, display
display(Image(filename='sales_by_region.png'))
```

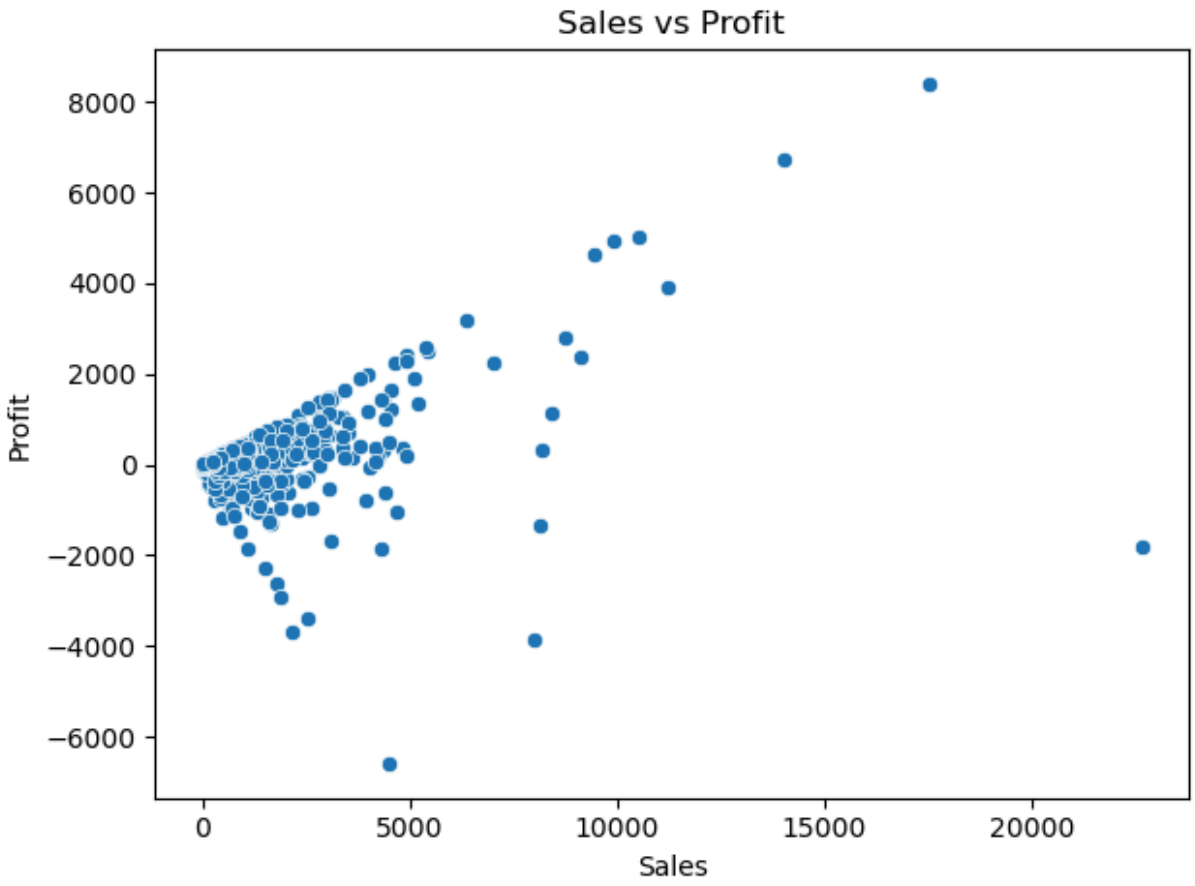


In [77]:
```
# 📊 One-Page Summary Report

# Chart 1: Sales by Region
plt.figure(figsize=(8,8))
```

```
plt.pie(
    sales_region.values,
    labels=sales_region.index,
    autopct='%1.1f%%'
)
plt.title("Sales Distribution by Region")
plt.tight_layout()
plt.savefig("sales_by_region.png", bbox_inches='tight')  # save BEFORE show
plt.show()
plt.close()

# Chart 2: Profit by Category
plt.figure(figsize=(10,5))

plt.bar(profit_category.index, profit_category.values)
plt.title("Profit by Category")
plt.xlabel("Category")
plt.ylabel("Profit")
plt.tight_layout()
plt.savefig("profit_by_category.png", bbox_inches='tight')  # save BEFORE show
plt.show()
plt.close()

# Chart 3
sns.scatterplot(x='Sales', y='Profit', data=df)
plt.title("Sales vs Profit")
plt.tight_layout()
plt.savefig("sales_vs_profit.png", bbox_inches='tight')  # save BEFORE show
plt.show()
plt.close()
```


Sales Distribution by Region
East
Central
29.5%
21.8%
17.1%
31.6%
South
West


Profit by Category
140000
120000
100000
80000
60000
40000
20000
0
Profit
Furniture
Office Supplies
Technology
Category



🔍 Key Insights

- West region contributes the highest share of sales.
- Technology category generates the maximum profit.
- Some categories show high sales but low profitability.
- Sales and profit are positively correlated.

In [79]:
```
!jupyter nbconvert --to notebook --execute YourNotebook.ipynb --inplace
```

```
This application is used to convert notebook files (*.ipynb)
        to various other formats.

        WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options
=======
The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.
To see all configurable class-options for some <cmd>, use:
    <cmd> --help-all

--debug
    set log level to logging.DEBUG (maximize logging output)
    Equivalent to: [--Application.log_level=10]
--show-config
    Show the application's configuration (human-readable format)
    Equivalent to: [--Application.show_config=True]
--show-config-json
    Show the application's configuration (json format)
    Equivalent to: [--Application.show_config_json=True]
--generate-config
    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]
-y
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]
--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]
--allow-errors
    Continue notebook execution even if one of the cells throws an error and include
the error message in the cell output (the default behaviour is to abort conversion).
This flag is only relevant if '--execute' was specified, too.
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]
--stdin
    read a single notebook file from stdin. Write the resulting notebook with defaul
t basename 'notebook.*'
    Equivalent to: [--NbConvertApp.from_stdin=True]
--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]
--inplace
    Run nbconvert in place, overwriting the existing notebook (only
            relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_for
mat=notebook --FilesWriter.build_directory=]
--clear-output
    Clear output of current file and save in place,
            overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_for
mat=notebook --FilesWriter.build_directory= --ClearOutputPreprocessor.enabled=True]
--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each cel
l).
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_for
```

```
mat=notebook --FilesWriter.build_directory= --CoalesceStreamsPreprocessor.enabled=Tr
ue]
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True --TemplateExporter.
exclude_output_prompt=True]
--no-input
    Exclude input cells and output prompts from converted document.
            This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True --TemplateExporte
r.exclude_input=True --TemplateExporter.exclude_input_prompt=True]
--allow-chromium-download
    Whether to allow downloading chromium if no suitable version is found on the sys
tem.
    Equivalent to: [--WebPDFExporter.allow_chromium_download=True]
--disable-chromium-sandbox
    Disable chromium security sandbox when converting to PDF..
    Equivalent to: [--WebPDFExporter.disable_sandbox=True]
--show-input
    Shows code input. This flag is only useful for dejavu users.
    Equivalent to: [--TemplateExporter.exclude_input=False]
--embed-images
    Embed the images as base64 dataurls in the output. This flag is only useful for
the HTML/WebPDF/Slides exports.
    Equivalent to: [--HTMLExporter.embed_images=True]
--sanitize-html
    Whether the HTML in Markdown cells and cell outputs should be sanitized..
    Equivalent to: [--HTMLExporter.sanitize_html=True]
--log-level=<Enum>
    Set the log level by value or name.
    Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR', 'CRITI
CAL']
    Default: 30
    Equivalent to: [--Application.log_level]
--config=<Unicode>
    Full path of a config file.
    Default: ''
    Equivalent to: [--JupyterApp.config_file]
--to=<Unicode>
    The export format to be used, either one of the built-in formats
            ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf',
'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
            or a dotted object name that represents the import path for an
            ``Exporter`` class
    Default: ''
    Equivalent to: [--NbConvertApp.export_format]
--template=<Unicode>
    Name of the template to use
    Default: ''
    Equivalent to: [--TemplateExporter.template_name]
--template-file=<Unicode>
    Name of the template file to use
    Default: None
    Equivalent to: [--TemplateExporter.template_file]
--theme=<Unicode>
    Template specific theme(e.g. the name of a JupyterLab CSS theme distributed
```

```
    as prebuilt extension for the lab template)
    Default: 'light'
    Equivalent to: [--HTMLExporter.theme]
--sanitize_html=<Bool>
    Whether the HTML in Markdown cells and cell outputs should be sanitized.This
    should be set to True by nbviewer or similar tools.
    Default: False
    Equivalent to: [--HTMLExporter.sanitize_html]
--writer=<DottedObjectName>
    Writer class used to write the
                                        results of the conversion
    Default: 'FilesWriter'
    Equivalent to: [--NbConvertApp.writer_class]
--post=<DottedOrNone>
    PostProcessor class used to write the
                                        results of the conversion
    Default: ''
    Equivalent to: [--NbConvertApp.postprocessor_class]
--output=<Unicode>
    Overwrite base name use for output files.
                Supports pattern replacements '{notebook_name}'.
    Default: '{notebook_name}'
    Equivalent to: [--NbConvertApp.output_base]
--output-dir=<Unicode>
    Directory to write output(s) to. Defaults
                                  to output to the directory of each notebook. To re
cover
                                  previous default behaviour (outputting to the curr
ent
                                  working directory) use . as the flag value.
    Default: ''
    Equivalent to: [--FilesWriter.build_directory]
--reveal-prefix=<Unicode>
    The URL prefix for reveal.js (version 3.x).
            This defaults to the reveal CDN, but can be any url pointing to a copy
            of reveal.js.
            For speaker notes to work, this must be a relative path to a local
            copy of reveal.js: e.g., "reveal.js".
            If a relative path is given, it must be a subdirectory of the
            current directory (from which the server is run).
            See the usage documentation
            (https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-html-sl
ideshow)
            for more details.
    Default: ''
    Equivalent to: [--SlidesExporter.reveal_url_prefix]
--nbformat=<Enum>
    The nbformat version to write.
            Use this to downgrade notebooks.
    Choices: any of [1, 2, 3, 4]
    Default: 4
    Equivalent to: [--NotebookExporter.nbformat_version]

Examples
--------
```

```
    The simplest way to use nbconvert is

            > jupyter nbconvert mynotebook.ipynb --to html

            Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'not
ebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf'].

            > jupyter nbconvert --to latex mynotebook.ipynb

            Both HTML and LaTeX support multiple output templates. LaTeX includes
            'base', 'article' and 'report'.  HTML includes 'basic', 'lab' and
            'classic'. You can specify the flavor of the format used.

            > jupyter nbconvert --to html --template lab mynotebook.ipynb

            You can also pipe the output to stdout, rather than a file

            > jupyter nbconvert mynotebook.ipynb --stdout

            PDF is generated via latex

            > jupyter nbconvert mynotebook.ipynb --to pdf

            You can get (and serve) a Reveal.js-powered slideshow

            > jupyter nbconvert myslides.ipynb --to slides --post serve

            Multiple notebooks can be given at the command line in a couple of
            different ways:

            > jupyter nbconvert notebook*.ipynb
            > jupyter nbconvert notebook1.ipynb notebook2.ipynb

            or you can specify the notebooks list in a config file, containing::

                c.NbConvertApp.notebooks = ["my_notebook.ipynb"]

            > jupyter nbconvert --config mycfg.py

To see all available configurables, use `--help-all`.
```

```
[NbConvertApp] WARNING | pattern 'YourNotebook.ipynb' matched no files
```

In [80]:
```
jupyter nbconvert sales.ipynb --to html
```

```
  Cell In[80], line 1
    jupyter nbconvert YourNotebook.ipynb --to html
            ^
SyntaxError: invalid syntax
```

In [ ]: